DESENVOLVIMENTO DE SOFTWARE VISUAL · 2026

PokéDraft

Manual Técnico do Projeto

Documento de referência do sistema: regras do domínio, arquitetura, função de cada arquivo, estrutura completa do banco de dados, conceitos de C# usados no código e o passo a passo para implementar novas funcionalidades.

Versão do documento: 1.0 · Base de código: 1ª entrega (aula 28/08) — solução criada, banco modelado, primeira migration aplicada.
Repositório: github.com/eduardowoz/PokeDraft-DesenvolvimentoSoftwereVIsual

Índice

1. **O sistema** — o que é, glossário, regras do jogo, fluxo de uma partida
2. **Arquitetura** — stack, MVC, caminho de uma requisição, responsabilidades
3. **Ambiente** — pré-requisitos, setup, comandos, problemas comuns
4. **Mapa completo de arquivos** — o que cada arquivo faz, um por um

   4.1 Raiz · 4.2 Configuração · 4.3 Controllers · 4.4 Models · 4.5 Data · 4.6 Migrations · 4.7 Views · 4.8 wwwroot
5. **O banco de dados** — diagrama, ficha de cada tabela, índices, regras de exclusão
6. **Conceitos de C# e EF Core** que aparecem no nosso código
7. **Receita completa** — como criar uma funcionalidade do zero (exemplo real)
8. **Convenções do time** — nomes, pastas, commits
9. **Git e Trello** — fluxo de trabalho
10. **Cronograma** e divisão sugerida de tarefas

1. O sistema

1.1 O que é

PokéDraft é um sistema web de entretenimento que simula uma batalha Pokémon entre **duas pessoas**. Diferente de um jogo tradicional, o jogador não escolhe livremente qualquer Pokémon: ele precisa **montar um time dentro de um orçamento**, a partir de um conjunto de Pokémon sorteado aleatoriamente e **compartilhado com o adversário**. Isso cria disputa: se você não pegar um Pokémon forte, o oponente pega.

Público-alvo: qualquer pessoa que goste de Pokémon, batalhas e um componente de sorte/estratégia.

1.2 Glossário do domínio

Termos que aparecem o tempo todo no código. Todo mundo do grupo precisa usar as mesmas palavras.

Termo	Significado no nosso sistema
`Draft`	A fase de montagem do time, antes da batalha. Envolve banir, comprar e vender.
`Sample (pool)`	Um "pacote" de Pokémon sorteados aleatoriamente. São gerados vários samples por partida, e os **dois jogadores enxergam os mesmos**. No código: tabela `PokemonNoPool`, campo `NumeroSample`.
`Budget / Pontos`	O orçamento de cada jogador para comprar Pokémon. Definido na `Configuracao` (campo `BudgetPontos`).
`Custo`	Quanto um Pokémon ou item "vale" em pontos. Campo `Custo` em `Pokemon` e `Item`.
`Ban`	Remover um Pokémon do pool para que **ninguém** possa comprá-lo. Não custa pontos.
`Venda`	Devolver um Pokémon já comprado e recuperar **parte** dos pontos (multiplicado pelo `FatorVenda`, ex.: 0.5 = metade).
`Overall`	A "nota" de força de um Pokémon já montado. É o número que decide quem vence cada duelo.
`IV`	*Individual Values* — valores de 0 a 31 por status, sorteados. Pequeno ajuste fino na força.
`Natureza`	Característica do Pokémon que aumenta um status e reduz outro. Sorteada (não escolhida).
`Tipagem`	Os 18 tipos elementais (Fogo, Água, ...). Alguns têm vantagem sobre outros na batalha.
`Confronto`	Um duelo 1×1 dentro da batalha simulada. Uma batalha tem vários confrontos.

1.3 As regras do jogo, em detalhe

a) Draft de Pokémon

- O sistema sorteia **vários samples** de Pokémon (quantidade e tamanho definidos na `Configuracao`).
- Os dois jogadores veem **os mesmos samples** — o que um pegar, o outro não pega mais.
- Cada jogador pode **banir** Pokémon (limite definido em `BansPorJogador`). Pokémon banido sai de jogo para os dois.
- Cada jogador pode **comprar** Pokémon, gastando pontos do seu budget.
- Quando os pontos acabam, o jogador **não pode comprar mais** — a menos que venda alguém.
- **Venda:** devolve um Pokémon do time e recupera `Custo × FatorVenda` pontos (sempre menos do que pagou).

b) Draft de itens

- Existe **um único sample de itens** por partida (não vários, como nos Pokémon), também compartilhado.
- Cada item pode ser **equipado em um Pokémon do time**, somando `BonusOverall` ao overall dele.
- Um Pokémon carrega no máximo um item (campo `ItemId` em `EscolhaPokemon`, que é opcional).

c) Cálculo do overall

O overall tem duas partes:

Parte	Entra no cálculo	Onde fica guardado
Overall base calculado no fim do draft	Status base do Pokémon + bônus do item equipado + natureza + IVs	`EscolhaPokemon.OverallBase`
Overall de confronto calculado na hora da batalha	Overall base + extras por: vantagem de tipagem contra o adversário daquele duelo, velocidade e tipo de ataque	`ConfrontoBatalha.OverallA` e `OverallB`

> Por que separar os dois? Porque o overall base é **fixo** (não depende do oponente), mas a vantagem de tipagem só existe **contra um adversário específico**. O mesmo Pokémon pode ter overall de confronto diferente dependendo de quem ele enfrenta.

d) Simulação da batalha

1. Antes de simular, cada jogador define a **ordem** do seu time (campo `OrdemNaBatalha`).
2. Os times se enfrentam **1×1 na ordem**: o 1º do jogador A contra o 1º do jogador B, o 2º contra o 2º, e assim por diante.
3. Em cada confronto, calcula-se o overall de confronto dos dois lados. Quem tiver o maior, vence e ganha **1 ponto** para o seu jogador.
4. Ao final, quem tiver **mais pontos** vence a batalha.
5. **Empate** (ex.: 3–3): vence quem tiver a **maior média de overall** do time.

e) O que fica fora do escopo

- Mais de uma geração de Pokémon.
- Draft de golpes (moveset) — os Pokémon não escolhem ataques.
- Escolher IVs e Naturezas manualmente — o sistema sorteia.

1.4 Fluxo completo de uma partida

```
1. CRIAR PARTIDA
   Escolher os 2 jogadores + a configuração (regras)
   → cria 1 registro em Partidas com Status = "Configuracao"

2. SORTEAR OS POOLS
   Sortear N samples de Pokémon + o sample único de itens
   → cria vários registros em PokemonNoPool e ItensNoPool
   → Partida.Status vira "Draft"

3. DRAFT  (repete até os dois times ficarem prontos)
   ├─ banir Pokémon      → PokemonNoPool.Banido = true      + MovimentacaoDraft
   ├─ comprar Pokémon    → cria EscolhaPokemon, gasta pontos  + MovimentacaoDraft
   ├─ vender Pokémon     → EscolhaPokemon.Vendida = true, devolve pontos + MovimentacaoDraft
   └─ equipar item       → EscolhaPokemon.ItemId = X        + MovimentacaoDraft

4. ORDENAR O TIME
   Cada jogador define EscolhaPokemon.OrdemNaBatalha (1, 2, 3...)
   → Partida.Status vira "Batalha"

5. SIMULAR A BATALHA
   Para cada posição: calcula overall dos dois lados e decide o vencedor
   → cria 1 registro em ConfrontosBatalha por duelo
   → soma os pontos em Partida.PontosJogadorA / PontosJogadorB
   → define Partida.VencedorJogadorId (empate: desempata pela média do overall)
   → Partida.Status vira "Finalizada"
```

Repare que praticamente toda ação do jogador gera **dois** efeitos: muda o estado atual (ex.: cria a escolha) **e** registra a movimentação no histórico. Isso permite reconstruir o saldo de pontos e mostrar o log do draft na tela.

2. Arquitetura

2.1 Stack e por que cada escolha

Camada	Tecnologia	Papel e justificativa
Backend	C# · ASP.NET Core MVC · .NET 8	Roda no servidor. Guarda as **regras do jogo** (validação de saldo, sorteio, cálculo de overall). É o exigido pela disciplina. Versão 8 porque é LTS e é a usada nas aulas.
API	Controllers REST retornando JSON	Expõe os dados para o frontend via `GET/POST/PUT/DELETE`. É o padrão CRUD pedido no planejamento.
Banco	SQLite + Entity Framework Core 8	O banco inteiro é **um arquivo** (`pokedraft.db`). Ninguém precisa instalar nem configurar servidor de banco — crítico num grupo de 4 máquinas diferentes. O EF Core traduz classes C# em tabelas.
Frontend	HTML · CSS · JavaScript (`fetch`)	Sem framework (React/Angular). Views em Razor (`.cshtml`) e o JS chama a nossa própria API.
Dados	PokéAPI (pokeapi.co)	API pública e gratuita com status, tipos e sprites. Vamos **importar** para o nosso banco (não consultar em tempo real, para a tela não depender da internet).
Apoio	Git + GitHub · Trello	Versionamento e quadro Kanban de tarefas.

> **Mudança em relação ao planejamento original:** o documento previa **MySQL**. Trocamos por **SQLite**. Motivo: MySQL exigiria cada integrante instalar e configurar um servidor, criar usuário/senha e manter a connection string diferente em cada máquina — muita chance de "na minha máquina funciona". Com SQLite o banco é gerado por comando e é idêntico para todos. O código C# é praticamente o mesmo: se um dia quisermos trocar, muda-se o pacote e uma linha no `Program.cs`.

2.2 O padrão MVC

MVC divide o sistema em três responsabilidades. A analogia clássica é um restaurante:

Peça	Analogia	No nosso projeto
Model	A cozinha e os ingredientes	As classes em `Models/Entities/` e o acesso ao banco em `Data/`. São os dados e as regras.
Controller	O garçom	As classes em `Controllers/`. Recebem a requisição, chamam o Model e devolvem a resposta (uma View ou um JSON).
View	O prato servido na mesa	Os arquivos `.cshtml` em `Views/`. O HTML que o usuário enxerga.

> **Regra prática:** o Controller **nunca** monta HTML. A View **nunca** acessa o banco. E as regras do jogo (cálculo de overall, validação de saldo) não ficam espalhadas na View — ficam no backend.

2.3 Duas formas de ligar frontend e backend

Abordagem	Como funciona	Onde usamos
View Razor clássica	O Controller já devolve o HTML pronto com `return View(dados)`.	Páginas simples e estáticas: a home, telas de listagem sem interação pesada.
API + JavaScript dinâmica	O Controller devolve **JSON**; o JavaScript no navegador chama com `fetch()` e monta/atualiza a tela sem recarregar a página.	A tela do draft, que muda a cada ban/compra/venda sem recarregar. É o foco do projeto.

2.4 O caminho completo de uma requisição

Exemplo: o jogador clica em "Comprar" um Pokémon na tela do draft.

```
NAVEGADOR
   │  usuário clica no botão "Comprar"
   ▼
JavaScript  (wwwroot/js/site.js)
   │  fetch("/api/draft/comprar", { method:"POST", headers:{...}, body: JSON })
   ▼
ROTEAMENTO ASP.NET  (configurado em Program.cs)
   │  descobre qual Controller e qual método atende essa URL + verbo
   ▼
CONTROLLER  (Controllers/DraftController.cs)
   │  valida a entrada, aplica a regra (o jogador tem saldo? o Pokémon está livre?)
   ▼
DbContext  (Data/PokeDraftDbContext.cs)
   │  _context.EscolhasPokemon.Add(...);  _context.SaveChanges();
   ▼
EF CORE
   │  traduz o objeto C# em comandos SQL
   ▼
SQLite  (arquivo pokedraft.db)
   │  INSERT INTO EscolhasPokemon (...)  →  grava
   ▲
   │  ... e o caminho volta:
CONTROLLER devolve  return Ok(novoEstado)   →  vira JSON
   ▲
JavaScript recebe o JSON e atualiza a tela (saldo, lista, time)
```

Entender esse caminho é o que permite **debugar**: quando algo não funciona, você consegue perguntar "parou em qual etapa?" — o clique disparou? o fetch saiu? a rota bateu? o controller rodou? o banco gravou?

2.5 O que pode e o que não pode ir em cada pasta

Pasta	Deve conter	Não deve conter
`Models/Entities/`	Classes que representam tabelas: propriedades e relacionamentos.	Acesso a banco, chamadas HTTP, lógica de tela.
`Models/Enums/`	Listas de valores fixos.	Qualquer lógica.
`Data/`	O `DbContext` e configuração de mapeamento.	Regras do jogo.
`Controllers/`	Receber requisição, validar, chamar o Model, devolver resposta.	HTML montado em string; cálculos longos (esses vão para uma classe de serviço).
`Views/`	HTML com Razor: laços, condições, exibição de dados.	Consulta ao banco, regra de negócio.
`wwwroot/`	CSS, JS, imagens, sprites — arquivos que o navegador baixa.	Nada de C#. Nada sensível: **tudo aqui é público**.
`Migrations/`	Arquivos gerados pelo `dotnet ef`.	Edição manual (salvo caso raro e combinado com o grupo).

3. Ambiente de desenvolvimento

3.1 Pré-requisitos (uma vez por máquina)

Ferramenta	Onde baixar	Como conferir
.NET SDK 8.0	dotnet.microsoft.com/download/dotnet/8.0	`dotnet --version` → deve mostrar `8.x`
Git	git-scm.com/downloads	`git --version`
Visual Studio Code	code.visualstudio.com	Instalar as extensões **C#** e **C# Dev Kit**
dotnet-ef (CLI do EF Core)	`dotnet tool install --global dotnet-ef --version 8.*`	`dotnet ef --version`

> Se a máquina tiver **.NET 10** (ou outra versão nova) instalado junto com o 8, tudo bem: o arquivo `global.json` na raiz força o projeto a usar o SDK 8. É por isso que ele existe.

3.2 Rodando o projeto pela primeira vez

```
# 1. baixar o repositório
git clone https://github.com/eduardowoz/PokeDraft-DesenvolvimentoSoftwereVIsual.git
cd PokeDraft-DesenvolvimentoSoftwereVIsual

# 2. baixar os pacotes NuGet que o projeto usa
dotnet restore

# 3. criar o banco pokedraft.db a partir das migrations
dotnet ef database update --project PokeDraft.Web

# 4. rodar
dotnet run --project PokeDraft.Web
```

O terminal mostra algo como `Now listening on: http://localhost:5248` — abra esse endereço no navegador. Para parar: `Ctrl + C`.

3.3 Comandos do dia a dia

Comando	Para que serve
`dotnet build`	Compila. **Rode antes de todo commit** — se der erro aqui, não commita.
`dotnet run --project PokeDraft.Web`	Roda a aplicação.
`dotnet watch --project PokeDraft.Web`	Roda e **recarrega sozinho** quando você salva um arquivo. Ótimo para mexer em tela.
`dotnet ef migrations add Nome --project PokeDraft.Web`	Gera uma migration depois de mudar as entidades.
`dotnet ef database update --project PokeDraft.Web`	Aplica as migrations pendentes no seu `pokedraft.db`.
`dotnet ef migrations remove --project PokeDraft.Web`	Desfaz a **última** migration (só se ela ainda não foi aplicada/commitada).
`dotnet ef migrations list --project PokeDraft.Web`	Lista as migrations e quais já foram aplicadas.

3.4 Problemas comuns

Sintoma	Causa provável e solução
`dotnet ef` não é reconhecido	A ferramenta não está instalada nessa máquina. Rode `dotnet tool install --global dotnet-ef --version 8.*` e **abra um terminal novo**.
"no such table: ..." ao rodar	O banco não foi criado ou está desatualizado. Rode `dotnet ef database update --project PokeDraft.Web`.
Deu `git pull` e o sistema quebrou	Provavelmente vieram migrations novas. Rode `dotnet ef database update`. Se persistir, `dotnet restore` (pode ter pacote novo).
Banco "bagunçado" durante testes	Apague o arquivo `PokeDraft.Web/pokedraft.db` e rode `dotnet ef database update`. Ele é descartável — não tem nada que não possa ser recriado.
Erro de versão do SDK	Confira `dotnet --version`. Se não for 8.x, falta instalar o SDK 8 (o `global.json` exige ele).
CSS/JS não carrega na página	O arquivo precisa estar dentro de `wwwroot/`. Fora dela o navegador não acessa.
O `fetch` retorna 404	A rota do JavaScript está diferente da rota do Controller. Confira letra por letra, incluindo singular/plural.

4. Mapa completo de arquivos

4.0 Visão geral da árvore

```
PokeDraft-DesenvolvimentoSoftwereVIsual/          raiz do repositório
├─ PokeDraft.sln
├─ global.json
├─ README.md
├─ .gitignore
├─ .gitattributes
├─ docs/                                          documentação (este manual)
└─ PokeDraft.Web/                                  o projeto da aplicação
   ├─ PokeDraft.Web.csproj
   ├─ Program.cs
   ├─ appsettings.json
   ├─ appsettings.Development.json
   ├─ pokedraft.db                                o banco (não versionado)
   ├─ Properties/
   │  └─ launchSettings.json
   ├─ Controllers/
   │  └─ HomeController.cs
   ├─ Models/
   │  ├─ ErrorViewModel.cs
   │  ├─ Entities/                                10 classes = 10 tabelas
   │  │  ├─ Pokemon.cs
   │  │  ├─ Item.cs
   │  │  ├─ Jogador.cs
   │  │  ├─ Configuracao.cs
   │  │  ├─ Partida.cs
   │  │  ├─ PokemonNoPool.cs
   │  │  ├─ ItemNoPool.cs
   │  │  ├─ EscolhaPokemon.cs
   │  │  ├─ MovimentacaoDraft.cs
   │  │  └─ ConfrontoBatalha.cs
   │  └─ Enums/
   │     ├─ TipoPokemon.cs
   │     ├─ StatusPartida.cs
   │     └─ TipoMovimentacao.cs
   ├─ Data/
   │  └─ PokeDraftDbContext.cs
   ├─ Migrations/
   │  ├─ 20260828125500_InicialCreate.cs
   │  ├─ 20260828125500_InicialCreate.Designer.cs
   │  └─ PokeDraftDbContextModelSnapshot.cs
   ├─ Views/
   │  ├─ _ViewImports.cshtml
   │  ├─ _ViewStart.cshtml
   │  ├─ Home/
   │  │  ├─ Index.cshtml
   │  │  └─ Privacy.cshtml
   │  └─ Shared/
   │     ├─ _Layout.cshtml
   │     ├─ _Layout.cshtml.css
   │     ├─ Error.cshtml
   │     └─ _ValidationScriptsPartial.cshtml
   └─ wwwroot/
      ├─ favicon.ico
      ├─ css/site.css
      ├─ js/site.js
      └─ lib/                                     Bootstrap e jQuery
```

4.1 Arquivos da raiz

`PokeDraft.sln`

O que é: o arquivo de *solução*. Uma solução é apenas uma **lista de projetos** que formam o sistema.

Por que existe: hoje temos um único projeto (`PokeDraft.Web`), mas se um dia criarmos outro (ex.: um projeto separado só para os testes, ou um console para o importador da PokéAPI), ele é adicionado aqui e todos passam a compilar juntos com um `dotnet build` na raiz.

Você edita? Não à mão. Use `dotnet sln add caminho/do/projeto.csproj` .

`global.json`

O que é: trava a versão do SDK do .NET usada no projeto.

```
{ "sdk": { "version": "8.0.424", "rollForward": "latestFeature" } }
```

Por que existe: várias máquinas do grupo podem ter versões diferentes do .NET instaladas. Sem esse arquivo, um PC com .NET 10 compilaria com regras diferentes de um PC com .NET 8, gerando bugs que só acontecem "na máquina de fulano". Com ele, todo mundo usa o mesmo SDK.

O `rollForward` **:** permite usar uma versão 8.0.x mais nova se a exata (8.0.424) não estiver instalada — mas nunca pular para o 9 ou 10.

`README.md`

O que é: o documento de apresentação do repositório. É o que o GitHub mostra automaticamente na página inicial do projeto.

O que contém: descrição do sistema, tabela de tecnologias, pré-requisitos, como rodar, estrutura de pastas, modelo de dados, fluxo de Git e cronograma.

Importante: o README precisa ser mantido atualizado — é o primeiro lugar que o professor vai olhar na entrega final.

`.gitignore`

O que é: a lista do que o Git deve **ignorar**.

O que ignoramos e por quê:

Item	Motivo
`bin/ e obj/`	Resultado da compilação. São gerados por `dotnet build` e mudam a cada compilação — versionar isso encheria o repositório de conflitos inúteis.
`*.db`	O banco SQLite. Cada integrante gera o seu com `dotnet ef database update`. Se versionássemos, todo mundo teria conflito toda vez que testasse alguma coisa.
`.vs/ e .vscode/`	Configuração pessoal do editor de cada um.

> **O que NÃO é ignorado (e é essencial):** a pasta `Migrations/`. Ela é a "receita" para qualquer pessoa reconstruir o banco. Versionar as migrations e ignorar o `.db` é o que permite os 4 trabalharem sem pisar no banco um do outro.

`.gitattributes`

O que é: padroniza o caractere de "fim de linha" dos arquivos de texto.

Por que importa: Windows termina linhas com `CRLF` e Linux/Mac com `LF`. Sem esse arquivo, alguém salvar um arquivo em outro sistema faria o Git marcar o **arquivo inteiro** como modificado, mesmo sem ninguém ter mudado nada — poluindo os diffs e criando conflitos falsos.

4.2 Arquivos de configuração do projeto

`PokeDraft.Web/PokeDraft.Web.csproj`

O que é: a definição do projeto — para o .NET, é o arquivo mais importante depois do código.

O que declara:

Configuração	Significado
`Sdk="Microsoft.NET.Sdk.Web"`	É um projeto **web** (traz o ASP.NET Core junto).
`<TargetFramework>net8.0</>`	Compila para .NET 8.
`<Nullable>enable</>`	Liga a checagem de nulos: o compilador avisa quando você pode estar usando algo que é `null`. É por isso que aparecem `?` e `= null!` nas entidades.
`<ImplicitUsings>enable</>`	Importa automaticamente os `using` mais comuns (`System`, `System.Collections.Generic`, etc.), por isso os arquivos têm poucos `using` no topo.

Os pacotes NuGet instalados:

Pacote	Para que serve
`Microsoft.EntityFrameworkCore.Sqlite`	O "tradutor" do EF Core para SQLite. É o que faz `UseSqlite(...)` existir e o que gera o SQL do SQLite.
`Microsoft.EntityFrameworkCore.Design`	Necessário para **gerar migrations**. Só é usado em tempo de desenvolvimento (não vai para o servidor).
`Microsoft.EntityFrameworkCore.Tools`	Comandos do EF dentro do Visual Studio (Package Manager Console). Complementa o `dotnet ef`.

Todos travados na versão **8.0.10** — a mesma linha do .NET 8. Misturar EF 9 ou 10 com .NET 8 dá erro.

`PokeDraft.Web/Program.cs`

O que é: o ponto de entrada da aplicação. É a primeira coisa que roda. Divide-se em duas metades:

Metade 1 — registrar serviços (o que a aplicação sabe fazer):

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();   // habilita Controllers + Views (MVC)

// registra o nosso banco: EF Core usando SQLite,
// com a connection string que está no appsettings.json
builder.Services.AddDbContext<PokeDraftDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("PokeDraftDb")));
```

Metade 2 — montar o pipeline (por onde cada requisição passa, **na ordem**):

```
var app = builder.Build();

app.UseHttpsRedirection();   // http → https
app.UseStaticFiles();        // serve o que está em wwwroot/
app.UseRouting();            // descobre qual controller atende a URL
app.UseAuthorization();      // checa permissões (ainda não usamos)

app.MapControllerRoute(      // a rota padrão do MVC
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();                   // sobe o servidor
```

A rota padrão explicada: `{controller=Home}/{action=Index}/{id?}` significa que a URL `/Pokemon/Detalhes/25` chama o método `Detalhes(25)` da classe `PokemonController`. Se a URL for só `/`, usa os padrões: `HomeController.Index()`. O `?` em `id` quer dizer opcional.

Você vai mexer aqui quando: registrar um novo serviço (ex.: o `HttpClient` do importador da PokéAPI) ou uma classe de regra de negócio.

`PokeDraft.Web/appsettings.json`

O que é: o arquivo de configuração da aplicação, em JSON. O `Program.cs` lê valores daqui.

```
{
  "ConnectionStrings": {
    "PokeDraftDb": "Data Source=pokedraft.db"
  },
  "Logging": { ... },
  "AllowedHosts": "*"
}
```

A connection string: `Data Source=pokedraft.db` diz ao SQLite **qual arquivo** é o banco. O nome `"PokeDraftDb"` é o que aparece em `GetConnectionString("PokeDraftDb")` no `Program.cs` — os dois têm que bater.

Regra: nunca coloque senha ou chave de API aqui — este arquivo vai para o Git.

`PokeDraft.Web/appsettings.Development.json`

O que é: configurações que valem **apenas em desenvolvimento**, sobrescrevendo o `appsettings.json`.

Hoje só ajusta o nível de log. A ideia é poder ter logs detalhados na sua máquina e logs enxutos em produção, sem trocar código.

`PokeDraft.Web/Properties/launchSettings.json`

O que é: define **como o projeto é iniciado localmente** — portas, se abre o navegador sozinho e qual ambiente usar.

No nosso projeto: perfil `http` em `localhost:5248` e perfil `https` em `localhost:7223`. Os dois definem `ASPNETCORE_ENVIRONMENT=Development`, que é o que faz o `appsettings.Development.json` ser carregado.

Vale só na sua máquina — não afeta a aplicação publicada.

4.3 Controllers

`Controllers/HomeController.cs`

O que é: o controller da página inicial. Veio do template do `dotnet new mvc` e serve de **modelo** para os controllers que vamos criar.

O que tem dentro:

Membro	Função
`Index()`	Devolve a view `Views/Home/Index.cshtml` (a home do PokéDraft).
`Privacy()`	Devolve `Views/Home/Privacy.cshtml`. Página do template — pode ser removida ou reaproveitada depois.
`Error()`	Devolve a tela de erro com o identificador da requisição, para ajudar a rastrear o problema no log.
`ILogger<HomeController>`	Recebido no **construtor** por injeção de dependência. Serve para escrever logs.

Dois pontos importantes que se repetem em todo controller:

- A classe herda de `Controller` (para MVC com Views) ou de `ControllerBase` (para API pura, sem Views).
- O nome do arquivo define a rota: `HomeController` → URL `/Home`. O sufixo "Controller" é removido automaticamente.

Os controllers que vamos criar nas próximas aulas: `PokemonController` (04/09), `JogadorController`, `PartidaController`, `ConfiguracaoController` (11/09) e `DraftController` (02/10 e 09/10). Os de API vão usar `[ApiController]` e `[Route("api/[controller]")]`.

4.4 Models — as entidades do domínio

Cada arquivo em `Models/Entities/` é uma **classe C#** que o EF Core transforma em uma **tabela**. As propriedades viram colunas. Abaixo, a ficha de cada uma.

`Pokemon.cs` — catálogo de Pokémon

Representa: um Pokémon disponível no jogo, com os dados importados da PokéAPI. É "a carta" que aparece nos pools.

Propriedade	Tipo	Significado
`Id`	int	Chave primária, gerada pelo banco.
`PokeApiId`	int	O id do Pokémon **na PokéAPI**. Tem índice único: impede o importador de cadastrar o mesmo Pokémon duas vezes.
`Nome`	string	Nome do Pokémon (máx. 60 caracteres).
`Tipo1`	TipoPokemon	Tipo principal. Obrigatório.
`Tipo2`	TipoPokemon?	Segundo tipo. **Opcional** — muitos Pokémon têm só um.
`HpBase, AtaqueBase, DefesaBase, AtaqueEspecialBase, DefesaEspecialBase, VelocidadeBase`	int	Os seis status base vindos da PokéAPI. São a matéria-prima do cálculo de overall.
`TotalStatusBase`	int	Soma dos seis. Guardado pronto para não ter que somar toda hora — útil para ranquear e para definir o `Custo`.
`Geracao`	int	Geração do Pokémon. Como o escopo é uma geração só, serve de filtro.
`SpriteUrl`	string?	URL da imagem do Pokémon, para mostrar na tela.
`Custo`	int	Quanto custa em pontos adicionar este Pokémon ao time.
`AparicoesEmPools Escolhas`	coleções	Propriedades de **navegação**: não viram colunas. Permitem escrever `pokemon.Escolhas` no C# para acessar os registros relacionados.

`Item.cs` — catálogo de itens

Representa: um item de batalha que pode ser equipado em um Pokémon.

Propriedade	Tipo	Significado
`Id`	int	Chave primária.
`Nome`	string	Nome do item (máx. 80).
`Descricao`	string?	O que o item faz, em texto (máx. 400).
`Categoria`	string?	Agrupamento livre (ex.: "Berry", "Held Item"). Ajuda a filtrar na tela.
`BonusOverall`	int	**Quanto o item soma ao overall** do Pokémon que o carrega.
`SpriteUrl`	string?	Imagem do item.
`Custo`	int	Custo em pontos para adquirir.

`Jogador.cs` — participante

Representa: uma pessoa que joga. Cada partida tem exatamente dois.

Propriedade	Tipo	Significado
`Id`	int	Chave primária.
`Nome`	string	Nome do jogador (máx. 60).
`CriadoEm`	DateTime	Data de cadastro. Preenchido automaticamente com a data/hora atual.

Entidade simples de propósito — não temos login/senha no escopo.

`Configuracao.cs` — as regras do draft

Representa: um "preset" reutilizável de regras. Em vez de fixar os números no código, eles ficam no banco e podem ser ajustados sem recompilar. Uma partida aponta para uma configuração.

Propriedade	Padrão	Significado
`Nome`	—	Nome do preset (ex.: "Padrão", "Rápido"). Tem índice único.
`BudgetPontos`	100	Pontos que cada jogador recebe.
`QuantidadeSamples`	5	Quantos samples de Pokémon são sorteados.
`PokemonPorSample`	6	Quantos Pokémon cada sample contém.
`TamanhoTime`	6	Máximo de Pokémon no time de cada jogador.
`BansPorJogador`	2	Quantos banimentos cada um pode fazer.
`FatorVenda`	0.5	Multiplicador aplicado ao custo na venda. 0.5 = recupera metade.
`CriadaEm`	agora	Data de criação do preset.

Já existe uma configuração **"Padrão"** cadastrada automaticamente (ver seção 5.5).

`Partida.cs` — a entidade central

Representa: uma disputa entre dois jogadores. É o "hub" do sistema: quase todas as outras tabelas apontam para ela.

Propriedade	Tipo	Significado
`Nome`	string?	Rótulo opcional da partida.
`Status`	StatusPartida	Em que etapa está: Configuracao → Draft → Batalha → Finalizada. **Controla o que o sistema permite fazer** (ex.: não dá para comprar Pokémon se o status já é "Batalha").
`CriadaEm / FinalizadaEm`	DateTime / ?	Início e fim. `FinalizadaEm` fica nulo até a batalha terminar.
`ConfiguracaoId`	int (FK)	Qual conjunto de regras esta partida usa.
`JogadorAId / JogadorBId`	int (FK)	Os dois participantes.
`VencedorJogadorId`	int? (FK)	Quem venceu. Nulo até finalizar.
`PontosJogadorA / B`	int	O **placar** da batalha: quantos confrontos cada lado venceu.
`PokemonNoPool, ItensNoPool, Escolhas, Movimentacoes, Confrontos`	coleções	Navegação para tudo que pertence a esta partida.

`PokemonNoPool.cs` — os Pokémon sorteados

Representa: a presença de um Pokémon em um sample de uma partida. Aqui vive o **estado do draft**: se foi banido, se já foi comprado e por quem.

Propriedade	Tipo	Significado
`PartidaId`	int (FK)	De qual partida é este pool.
`PokemonId`	int (FK)	Qual Pokémon do catálogo foi sorteado.
`NumeroSample`	int	Em qual sample (1, 2, 3...) ele apareceu.
`Banido`	bool	Se `true`, ninguém pode comprá-lo.
`BanidoPorJogadorId`	int?	Quem baniu (para o histórico e para contar o limite de bans).
`CompradoPorJogadorId`	int?	Quem comprou. Enquanto for nulo, o Pokémon está **disponível**.

Por que não guardar isso direto na tabela `Pokemon`? Porque o mesmo Pikachu pode estar em várias partidas ao mesmo tempo, banido numa e comprado em outra. O estado pertence à partida, não ao catálogo.

`ItemNoPool.cs` — os itens disponíveis

Representa: um item no sample único de itens de uma partida. Mesma lógica do `PokemonNoPool`, mas mais simples (itens não são banidos e há um só sample).

Propriedade	Tipo	Significado
`PartidaId`	int (FK)	De qual partida.
`ItemId`	int (FK)	Qual item do catálogo.
`AdquiridoPorJogadorId`	int?	Quem pegou. Nulo = ainda disponível.

`EscolhaPokemon.cs` — um Pokémon no time

Representa: um Pokémon que um jogador **comprou e colocou no time**, já com item, natureza, IVs, posição de batalha e overall calculado. É a entidade mais rica do sistema.

Propriedade	Tipo	Significado
`PartidaId / JogadorId`	int (FK)	De qual partida e de qual jogador é este Pokémon.
`PokemonId`	int (FK)	Qual Pokémon do catálogo.
`ItemId`	int? (FK)	Item equipado. Opcional — nem todo Pokémon carrega item.
`PontosGastos`	int	Quanto custou **no momento da compra**. Guardado aqui porque o custo do catálogo pode mudar depois — o histórico tem que refletir o que realmente aconteceu.
`OrdemNaBatalha`	int?	Posição na batalha (1 = enfrenta o 1º do adversário). Nulo enquanto o jogador não ordenou.
`Natureza`	string?	Natureza sorteada.
`IvHp, IvAtaque, IvDefesa, IvAtaqueEspecial, IvDefesaEspecial, IvVelocidade`	int	Os seis IVs sorteados (0 a 31 cada).
`OverallBase`	int	O overall calculado fora da batalha (status + item + natureza + IVs).
`Vendida`	bool	Se o jogador vendeu este Pokémon. **O registro não é apagado** — fica marcado, preservando o histórico.
`PontosRecebidosNaVenda`	int?	Quantos pontos voltaram na venda.

`MovimentacaoDraft.cs` — o histórico

Representa: o registro de **cada ação** feita no draft. É o "extrato bancário" da partida.

Propriedade	Tipo	Significado
`PartidaId / JogadorId`	int (FK)	De qual partida e quem fez a ação.
`Tipo`	TipoMovimentacao	Banimento, CompraPokemon, VendaPokemon, EquiparItem ou RemoverItem.
`PokemonId / ItemId`	int? (FK)	O que foi envolvido. Ambos opcionais porque depende do tipo da ação.
`VariacaoPontos`	int	Quanto mudou no saldo: **negativo** em compras, **positivo** em vendas, **zero** em bans.
`SaldoResultante`	int	O saldo do jogador logo após a ação. Facilita mostrar o extrato sem recalcular tudo.
`Descricao`	string?	Texto livre para exibir na tela (ex.: "Comprou Charizard por 25 pontos").
`Momento`	DateTime	Quando aconteceu — define a ordem cronológica do log.

Por que ter essa tabela? Ela é a entrega do dia 09/10 ("histórico de movimentações"), permite mostrar o log do draft na tela e é a forma de auditar/depurar quando um saldo aparecer errado.

`ConfrontoBatalha.cs` — um duelo

Representa: um confronto 1×1 da batalha simulada, com os overalls dos dois lados e quem venceu.

Propriedade	Tipo	Significado
`PartidaId`	int (FK)	De qual partida.
`Ordem`	int	Qual posição do confronto (1 = primeiros Pokémon de cada time).
`EscolhaPokemonAId` `EscolhaPokemonBId`	int (FK)	Os dois Pokémon que se enfrentaram. Apontam para `EscolhasPokemon`, não para o catálogo — porque importa o Pokémon **com item e IVs daquele jogador**.
`OverallA / OverallB`	int	Os overalls **finais** do confronto (já com os bônus de tipagem, velocidade e tipo de ataque).
`VencedorJogadorId`	int?	Quem venceu este duelo. Nulo em caso de empate no confronto.

Guardar os overalls calculados permite **mostrar o resultado detalhado** na tela ("Charizard 187 × 174 Blastoise") em vez de só o placar final.

Os enums — `Models/Enums/`

Um `enum` é uma lista fechada de valores possíveis. Usar enum em vez de `string` solta evita erro de digitação: o compilador não deixa escrever `StatusPartida.Batalh`.

Arquivo	Valores e uso
`TipoPokemon.cs`	Os 18 tipos: Normal, Fogo, Agua, Eletrico, Planta, Gelo, Lutador, Venenoso, Terrestre, Voador, Psiquico, Inseto, Pedra, Fantasma, Dragao, Sombrio, Aco, Fada. Usado em `Pokemon.Tipo1/Tipo2` e será a base da tabela de vantagem de tipagem.
`StatusPartida.cs`	`Configuracao(1) → Draft(2) → Batalha(3) → Finalizada(4)`. Controla o que é permitido em cada etapa.
`TipoMovimentacao.cs`	`Banimento(1)`, `CompraPokemon(2)`, `VendaPokemon(3)`, `EquiparItem(4)`, `RemoverItem(5)`.

> **`Models/ErrorViewModel.cs`**
>
> **O que é:** um *ViewModel* — uma classe feita só para **alimentar uma tela**, não para virar tabela. Este veio do template e carrega o `RequestId` mostrado na página de erro.
>
> **Guarde o conceito:** vamos criar vários ViewModels próprios (ex.: `DraftViewModel` com o pool + o time + o saldo juntos). Eles ficam em `Models/`, **fora** de `Entities/`, justamente porque não são tabelas.

4.5 Data — a ponte com o banco

> **`Data/PokeDraftDbContext.cs`**
>
> **O que é:** a classe mais importante da camada de dados. Herda de `DbContext` (do EF Core) e faz três coisas:
>
> **1. Declara as tabelas** — um `DbSet<T>` por entidade:
>
> ```
> public DbSet<Pokemon> Pokemons => Set<Pokemon>();
> public DbSet<Item> Itens => Set<Item>();
> public DbSet<Jogador> Jogadores => Set<Jogador>();
> // ... e assim por diante, 10 no total
> ```
>
> É por meio deles que se consulta e grava: `_context.Pokemons.ToList()`, `_context.Jogadores.Add(j)`.
>
> **2. Configura o mapeamento** no método `OnModelCreating`:
>
> - **Enums como texto:** `.HasConversion<string>()` faz o banco guardar `"Draft"` em vez de `2`. Fica muito mais fácil de ler abrindo o banco direto.
> - **Índices únicos:** em `Pokemon.PokeApiId` (evita importar duplicado) e `Configuracao.Nome`.
> - **Relacionamentos:** quem aponta para quem e o que acontece ao apagar (ver seção 5.4).
> - **Dado inicial:** a configuração "Padrão" (ver 5.5).
>
> **3. Recebe as opções por injeção de dependência** — o construtor recebe `DbContextOptions`, que o `Program.cs` preencheu com "use SQLite, arquivo pokedraft.db".
>
> > **Quando você mexe neste arquivo:** sempre que criar uma entidade nova (adicionar o `DbSet`) ou precisar configurar um relacionamento/índice. Depois de mexer, **gere uma migration**.

4.6 Migrations — o histórico do banco

Uma **migration** é um "commit do formato do banco". Ela guarda, em código, exatamente quais tabelas/colunas criar ou alterar. É isso que permite os 4 integrantes terem bancos idênticos sem passar arquivo `.db` um para o outro.

Arquivo	Conteúdo
`20260828125500_InicialCreate.cs`	A migration em si (476 linhas). Método `Up()` = **10** `CreateTable`, **21** `CreateIndex` e **1** `InsertData` (a configuração padrão). Método `Down()` = como desfazer tudo isso. O número no nome é a data/hora (2026-08-28 12:55:00) — é o que define a **ordem** de aplicação.
`20260828125500_InicialCreate.Designer.cs`	Metadados dessa migration: uma cópia do modelo no momento em que ela foi criada. Usado internamente pelo EF. **Não editar.**
`PokeDraftDbContextModelSnapshot.cs`	A "foto" do **estado atual** do modelo. Quando você roda `migrations add`, o EF compara suas classes com este snapshot para descobrir o que mudou e gerar só a diferença. **Não editar** — e cuidado: é o arquivo que mais dá conflito no Git quando duas pessoas criam migration ao mesmo tempo.

> **Regra de ouro das migrations em equipe:** avise no grupo antes de criar uma migration. Se duas pessoas criarem migrations em paralelo, o `ModelSnapshot` conflita. A solução é a segunda pessoa dar `git pull`, apagar a sua migration (`dotnet ef migrations remove`) e recriá-la em cima da versão do colega.

4.7 Views — as telas

Views são arquivos `.cshtml`: HTML normal com blocos de C# marcados por `@` (a sintaxe **Razor**). O servidor executa o C#, gera HTML puro e manda para o navegador.

Arquivo	Função
`Views/_ViewStart.cshtml`	Roda **antes** de cada view e define o layout padrão (`Layout = "_Layout"`). É por isso que nenhuma view precisa repetir isso.
`Views/_ViewImports.cshtml`	Os `@using` e *tag helpers* válidos em **todas** as views. Evita repetir imports em cada arquivo. Se você criar um namespace novo usado em muitas telas, adicione aqui.
`Views/Shared/_Layout.cshtml`	A **moldura** comum: `<html>` , `<head>` , menu de navegação, rodapé, e os `<link>` / `<script>` do Bootstrap, do `site.css` e do `site.js` . No meio dela existe `@RenderBody()` — o buraco onde o conteúdo de cada página é encaixado. **Mudou aqui, mudou em todas as telas.**
`Views/Shared/_Layout.cshtml.css`	CSS *com escopo*: vale só para o layout. O ASP.NET adiciona um identificador nos elementos para isolar esse estilo.
`Views/Shared/Error.cshtml`	Tela exibida quando estoura um erro não tratado. Recebe o `ErrorViewModel` .
`Views/Shared/_ValidationScriptsPartial.cshtml`	Bloco reutilizável com os scripts de validação de formulário no navegador. Uma view inclui com `@await Html.PartialAsync(...)` . O `_` no início do nome é a convenção para "arquivo parcial, não é uma página".
`Views/Home/Index.cshtml`	A home do PokéDraft. Hoje mostra título, descrição e o status da entrega.
`Views/Home/Privacy.cshtml`	Página de privacidade do template. Pode ser reaproveitada ou removida.

> **A convenção de pastas:** `Views/<NomeDoController>/<NomeDoMétodo>.cshtml` . Quando `PokemonController.Index()` chama `return View()` , o ASP.NET procura automaticamente por `Views/Pokemon/Index.cshtml` . Se não achar, procura em `Views/Shared/` .

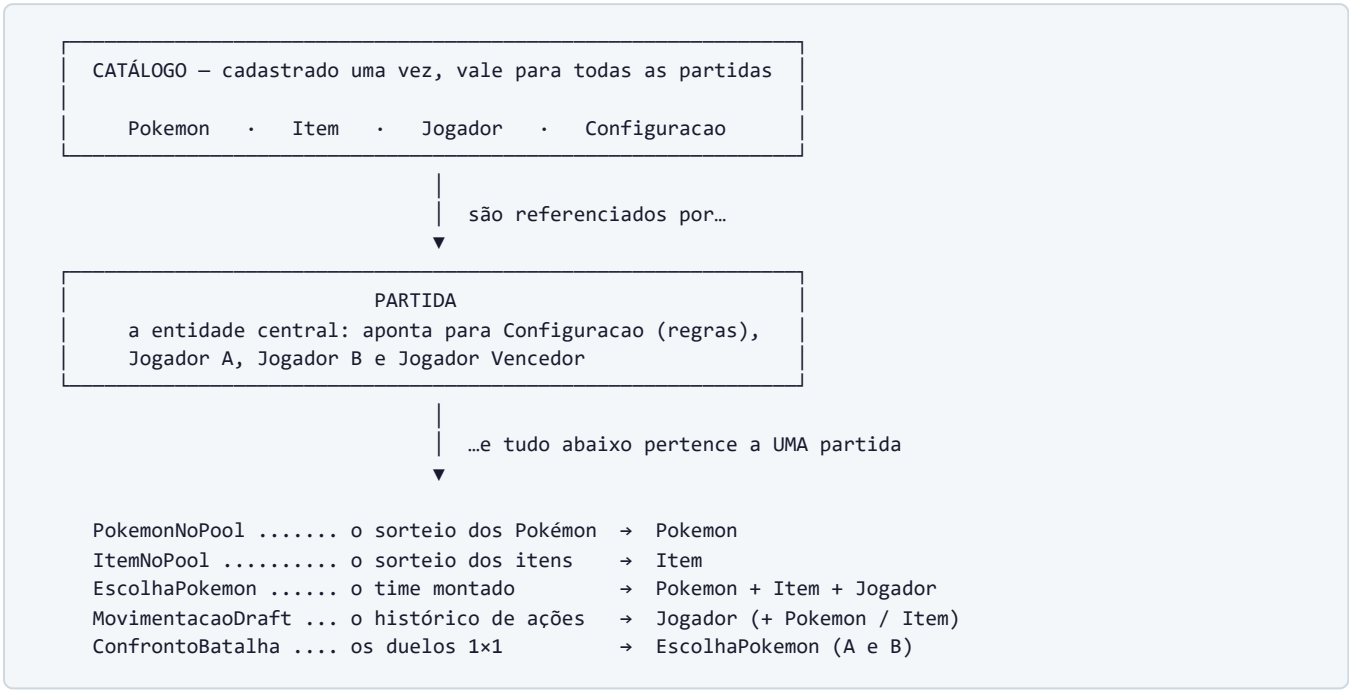
4.8 wwwroot — os arquivos públicos

Arquivo/pasta	Função
`wwwroot/css/site.css`	O CSS do site. **É aqui que vai o visual do PokéDraft** (entrega de 23/10).
`wwwroot/js/site.js`	O JavaScript. Hoje está praticamente vazio — é aqui que entram as funções com `fetch()` que conversam com a nossa API.
`wwwroot/favicon.ico`	Ícone da aba do navegador.
`wwwroot/lib/`	Bibliotecas de terceiros que vieram com o template: **Bootstrap** (grid e componentes visuais prontos), **jQuery** e os plugins de validação. Não editamos nada aqui.

> **Duas regras sobre o `wwwroot`:**
> 1. Só o que está aqui o navegador consegue baixar diretamente. CSS/JS colocado em outra pasta simplesmente não carrega.
> 2. Tudo aqui é **público**. Nunca coloque nada sensível — qualquer pessoa pode acessar digitando a URL.

5. O banco de dados

5.1 Diagrama de relacionamentos



Lendo o diagrama:

- **Catálogo** (Pokemon, Item, Jogador, Configuracao): dados que existem independentemente de qualquer partida. Preenchidos uma vez (importador da PokéAPI, cadastro de jogadores).
- **Partida** é o centro. Tudo que acontece durante o jogo pertence a uma partida.
- As tabelas de jogo **ligam** o catálogo à partida: `PokemonNoPool` diz "este Pokémon do catálogo está no pool desta partida"; `EscolhaPokemon` diz "este Pokémon do catálogo está no time deste jogador nesta partida".
- `ConfrontoBatalha` aponta para `EscolhaPokemon` (e não para `Pokemon`) porque o que luta é o Pokémon **já montado**, com item e IVs.

5.2 As tabelas, coluna por coluna

Tipos SQLite: `INTEGER` (números e booleanos), `TEXT` (textos, datas e enums), `REAL` (decimais). Toda tabela tem `Id INTEGER` como chave primária autoincremental.

Pokemons

Coluna	Tipo	Nulo?	Observação
`Id`	INTEGER	não	PK, autoincremento
`PokeApiId`	INTEGER	não	**índice único**
`Nome`	TEXT(60)	não	
`Tipo1`	TEXT(20)	não	enum como texto: "Fogo", "Agua"...
`Tipo2`	TEXT(20)	sim	nulo = tipagem simples
`HpBase, AtaqueBase, DefesaBase, AtaqueEspecialBase, DefesaEspecialBase, VelocidadeBase`	INTEGER	não	os seis status base
`TotalStatusBase`	INTEGER	não	soma pré-calculada
`Geracao`	INTEGER	não	
`SpriteUrl`	TEXT(300)	sim	
`Custo`	INTEGER	não	preço em pontos

Itens

Coluna	Tipo	Nulo?	Observação
`Id`	INTEGER	não	PK
`Nome`	TEXT(80)	não	
`Descricao`	TEXT(400)	sim	
`Categoria`	TEXT(60)	sim	
`BonusOverall`	INTEGER	não	soma ao overall do portador
`SpriteUrl`	TEXT(300)	sim	
`Custo`	INTEGER	não	

Jogadores

Coluna	Tipo	Nulo?	Observação
`Id`	INTEGER	não	PK
`Nome`	TEXT(60)	não	
`CriadoEm`	TEXT	não	data/hora ISO

Configuracoes

Coluna	Tipo	Nulo?	Observação
Id	INTEGER	não	PK
Nome	TEXT(80)	não	**índice único**
BudgetPontos	INTEGER	não	padrão 100
QuantidadeSamples	INTEGER	não	padrão 5
PokemonPorSample	INTEGER	não	padrão 6
TamanhoTime	INTEGER	não	padrão 6
BansPorJogador	INTEGER	não	padrão 2
FatorVenda	REAL	não	padrão 0.5
CriadaEm	TEXT	não	

Partidas

Coluna	Tipo	Nulo?	Observação
Id	INTEGER	não	PK
Nome	TEXT(100)	sim	
Status	TEXT(20)	não	"Configuracao" / "Draft" / "Batalha" / "Finalizada"
CriadaEm	TEXT	não	
FinalizadaEm	TEXT	sim	nulo até a batalha acabar
ConfiguracaoId	INTEGER	não	FK → Configuracoes
JogadorAId	INTEGER	não	FK → Jogadores
JogadorBId	INTEGER	não	FK → Jogadores
VencedorJogadorId	INTEGER	sim	FK → Jogadores
PontosJogadorA	INTEGER	não	placar
PontosJogadorB	INTEGER	não	placar

PokemonNoPool

Coluna	Tipo	Nulo?	Observação
Id	INTEGER	não	PK
PartidaId	INTEGER	não	FK → Partidas
PokemonId	INTEGER	não	FK → Pokemons
NumeroSample	INTEGER	não	índice composto com PartidaId
Banido	INTEGER	não	0 = false, 1 = true
BanidoPorJogadorId	INTEGER	sim	marcador (sem FK)
CompradoPorJogadorId	INTEGER	sim	nulo = disponível

ItensNoPool

Coluna	Tipo	Nulo?	Observação
`Id`	INTEGER	não	PK
`PartidaId`	INTEGER	não	FK → Partidas
`ItemId`	INTEGER	não	FK → Itens
`AdquiridoPorJogadorId`	INTEGER	sim	nulo = disponível

EscolhasPokemon

Coluna	Tipo	Nulo?	Observação
`Id`	INTEGER	não	PK
`PartidaId`	INTEGER	não	FK → Partidas
`JogadorId`	INTEGER	não	FK → Jogadores
`PokemonId`	INTEGER	não	FK → Pokemons
`ItemId`	INTEGER	sim	FK → Itens (item equipado)
`PontosGastos`	INTEGER	não	custo no momento da compra
`OrdemNaBatalha`	INTEGER	sim	1, 2, 3... nulo antes de ordenar
`Natureza`	TEXT(20)	sim	sorteada
`IvHp, IvAtaque, IvDefesa, IvAtaqueEspecial, IvDefesaEspecial, IvVelocidade`	INTEGER	não	0 a 31 cada
`OverallBase`	INTEGER	não	calculado no fim do draft
`Vendida`	INTEGER	não	registro preservado, não apagado
`PontosRecebidosNaVenda`	INTEGER	sim	

MovimentacoesDraft

Coluna	Tipo	Nulo?	Observação
`Id`	INTEGER	não	PK
`PartidaId`	INTEGER	não	FK → Partidas
`JogadorId`	INTEGER	não	FK → Jogadores
`Tipo`	TEXT(20)	não	"Banimento", "CompraPokemon"...
`PokemonId`	INTEGER	sim	FK → Pokemons
`ItemId`	INTEGER	sim	FK → Itens
`VariacaoPontos`	INTEGER	não	negativo/positivo/zero
`SaldoResultante`	INTEGER	não	saldo após a ação
`Descricao`	TEXT(200)	sim	texto para exibir
`Momento`	TEXT	não	ordem cronológica

ConfrontosBatalha

Coluna	Tipo	Nulo?	Observação
`Id`	INTEGER	não	PK
`PartidaId`	INTEGER	não	FK → Partidas
`Ordem`	INTEGER	não	posição do duelo
`EscolhaPokemonAId`	INTEGER	não	FK → EscolhasPokemon
`EscolhaPokemonBId`	INTEGER	não	FK → EscolhasPokemon
`OverallA`	INTEGER	não	overall final do lado A
`OverallB`	INTEGER	não	overall final do lado B
`VencedorJogadorId`	INTEGER	sim	nulo = empate no duelo

__EFMigrationsHistory

Tabela criada e usada **pelo próprio EF Core**. Guarda quais migrations já foram aplicadas neste banco. É como o EF sabe, ao rodar `database update`, que só precisa aplicar as novas. Nunca edite.

5.3 Índices

Um índice acelera buscas por uma coluna, ao custo de um pouco de espaço. Temos **21** no total:

Índice	Tipo	Motivo
`Pokemons.PokeApiId`	**único**	Garante que o importador não cadastre o mesmo Pokémon duas vezes, mesmo se rodar duas vezes.
`Configuracoes.Nome`	**único**	Não deixa criar dois presets com o mesmo nome.
`PokemonNoPool (PartidaId, NumeroSample)`	composto	A consulta mais frequente da tela do draft é "me dá o sample N da partida X". Este índice torna isso instantâneo.
`demais (18)`	automáticos	O EF cria um índice para **cada chave estrangeira**. Sem eles, buscar "todas as escolhas da partida 5" varreria a tabela inteira.

5.4 Regras de exclusão (o que acontece ao apagar)

Quando você apaga um registro, o que acontece com quem apontava para ele? Definimos isso explicitamente:

Relacionamento	Comportamento	Efeito prático
Partida → PokemonNoPool, ItensNoPool, EscolhasPokemon, MovimentacoesDraft, ConfrontosBatalha	**Cascade**	Apagar uma partida apaga **tudo que pertence a ela**. Faz sentido: pool, time e histórico não existem fora da partida.
Pokemon / Item / Jogador / Configuracao → qualquer coisa	**Restrict**	O banco **impede** apagar um Pokémon, item, jogador ou configuração que esteja sendo usado em alguma partida. Evita deixar registros órfãos e apagar histórico sem querer.
EscolhaPokemon → Item (equipado)	**SetNull**	Se um item for removido do catálogo, o Pokémon que o carregava simplesmente fica **sem item** — em vez de bloquear a exclusão ou apagar o Pokémon do time.
EscolhaPokemon → ConfrontoBatalha	**Restrict**	Não dá para apagar uma escolha que já participou de um confronto registrado.

> **Por que não deixar tudo em Cascade?** Além de perigoso (um `DELETE` errado apagaria meia base), o EF/SQL não permite **múltiplos caminhos de cascata** chegando na mesma tabela. Como `EscolhaPokemon` depende tanto de `Partida` quanto de `Jogador`, se os dois fossem Cascade daria erro ao criar o banco. Escolhemos: Partida cascateia, os demais restringem.

5.5 Dado inicial (seed)

A migration já insere uma `Configuracao` pronta, para que dê para criar uma partida sem cadastrar nada antes:

Campo	Valor
`Id / Nome`	1 / "Padrão"
`BudgetPontos`	100
`QuantidadeSamples × PokemonPorSample`	5 × 6 = 30 Pokémon no pool
`TamanhoTime`	6
`BansPorJogador`	2
`FatorVenda`	0.5 (recupera metade)

Esses valores são um **ponto de partida** para balancear. Depois de testar o draft, provavelmente vamos ajustar budget e custos.

6. Conceitos de C# e EF Core que aparecem no código

Tudo aqui está no código que já existe. Se algum trecho parecer estranho ao abrir um arquivo, provavelmente a explicação está nesta seção.

6.1 Propriedades com `get; set;`

```
public string Nome { get; set; } = null!;
```

Uma **propriedade** é um campo com controle de leitura (`get`) e escrita (`set`). O EF Core só transforma em coluna aquilo que for propriedade pública com `get` e `set` — por isso todas as entidades usam esse formato.

6.2 Nulos: `?` e `= null!`

Como o projeto tem `<Nullable>enable</>` , o compilador diferencia o que pode ou não ser nulo:

Escrita	Significado
`string Nome`	**Obrigatório.** No banco vira coluna `NOT NULL` .
`string? Descricao`	**Opcional.** Pode ser nulo; coluna aceita `NULL` .
`int? OrdemNaBatalha`	Número que **pode não ter valor ainda**. Diferente de zero: `null` = "não definido", `0` = "definido como zero".
`= null!`	Uma promessa ao compilador: "sei que parece nulo agora, mas o EF preenche antes de qualquer uso". Serve só para silenciar o aviso — não muda o banco.

6.3 Propriedades de navegação e `ICollection`

```
public int PartidaId { get; set; }              // vira coluna (a FK)
public Partida Partida { get; set; } = null!;   // navegação: NÃO vira coluna

public ICollection<EscolhaPokemon> Escolhas { get; set; } = new List<EscolhaPokemon>();
```

O par `PartidaId` + `Partida` é a convenção do EF: o `int` é a chave estrangeira real; o objeto é o atalho para navegar no C#. A coleção é o lado "muitos" do relacionamento — permite escrever `partida.Escolhas` em vez de fazer outra consulta.

Para a navegação vir preenchida do banco, é preciso pedir com `Include` :

```
var partida = _context.Partidas
    .Include(p => p.Escolhas)
        .ThenInclude(e => e.Pokemon)
    .FirstOrDefault(p => p.Id == id);
```

Sem o `Include` , `partida.Escolhas` vem vazia — não é bug, é o EF evitando carregar dados que você não pediu.

6.4 Enums e `HasConversion<string>()`

Por padrão o EF guardaria `StatusPartida.Draft` como o número `2` . Configuramos para guardar o texto `"Draft"` :

```
modelBuilder.Entity<Partida>()
    .Property(p => p.Status).HasConversion<string>().HasMaxLength(20);
```

Custa alguns bytes a mais, mas quem abrir o banco lê `"Finalizada"` em vez de decifrar `4`. Para um projeto de estudo, vale muito.

6.5 Data Annotations vs Fluent API

Duas formas de configurar o mapeamento. Usamos as duas:

Forma	Onde	Usamos para
Data Annotation	Atributo em cima da propriedade, na entidade: `[MaxLength(60)]`	Coisas simples e locais: tamanho máximo de texto.
Fluent API	Método `OnModelCreating`, no DbContext	Coisas que envolvem mais de uma entidade: relacionamentos, comportamento de exclusão, índices compostos, conversões, seed.

6.6 Injeção de dependência

```
public class PokemonController : ControllerBase
{
    private readonly PokeDraftDbContext _context;

    public PokemonController(PokeDraftDbContext context)   // chega pronto
    {
        _context = context;
    }
}
```

Você **nunca** escreve `new PokeDraftDbContext(...)` num controller. Como o `Program.cs` registrou o DbContext com `AddDbContext`, o ASP.NET cria e entrega a instância no construtor automaticamente — e cuida de fechar a conexão no fim da requisição. Mesmo padrão do `ILogger` no `HomeController`.

6.7 LINQ — consultando o banco

```
var disponiveis = _context.PokemonNoPool
    .Where(p => p.PartidaId == partidaId
             && !p.Banido
             && p.CompradoPorJogadorId == null)
    .Include(p => p.Pokemon)
    .OrderBy(p => p.NumeroSample)
    .ToList();
```

O EF traduz isso em `SELECT ... WHERE ... ORDER BY`. A consulta só é executada quando você chama `ToList()`, `FirstOrDefault()` ou similar.

Método	O que faz
`.Where(...)`	Filtra (vira `WHERE`).
`.OrderBy / .OrderByDescending`	Ordena.
`.Include(...)`	Traz junto os dados relacionados.
`.FirstOrDefault(...)`	Pega o primeiro ou `null` se não achar.
`.Any(...) / .Count(...)`	Existe algum? Quantos?
`.ToList()`	Executa e traz os resultados.

6.8 Gravando: `SaveChanges`

```
_context.Jogadores.Add(novoJogador);
_context.SaveChanges();     // só aqui o banco é realmente alterado
```

O DbContext **acumula** as mudanças em memória e grava tudo de uma vez no `SaveChanges()`, dentro de uma transação. Se der erro no meio, nada é gravado — importante para operações como "comprar Pokémon", que mexem em três tabelas ao mesmo tempo.

6.9 `async` / `await`

```
public async Task<IActionResult> Listar()
{
    var lista = await _context.Pokemons.ToListAsync();
    return Ok(lista);
}
```

Versões assíncronas liberam a *thread* enquanto o banco responde, permitindo o servidor atender mais requisições. Regra: se o método tem `await`, ele precisa ser `async` e devolver `Task<...>`. Vamos usar `async` nos controllers de API e no importador da PokéAPI.

7. Receita: criando uma funcionalidade do zero

Exemplo real e completo: **CRUD de Jogador** (parte da entrega de 11/09). Os mesmos seis passos servem para qualquer entidade.

Passo 1 — A entidade já existe?

`Models/Entities/Jogador.cs` já existe e já está no `DbContext` como `DbSet<Jogador> Jogadores`. Se fosse uma entidade nova, seria preciso criar a classe, adicionar o `DbSet` e gerar uma migration:

```
dotnet ef migrations add AdicionaEntidadeX --project PokeDraft.Web
dotnet ef database update --project PokeDraft.Web
```

Passo 2 — O Controller de API

Criar `Controllers/JogadorController.cs`:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using PokeDraft.Web.Data;
using PokeDraft.Web.Models.Entities;

namespace PokeDraft.Web.Controllers;

[ApiController]                         // marca como controller de API
[Route("api/[controller]")]             // rota base: /api/jogador
public class JogadorController : ControllerBase
{
    private readonly PokeDraftDbContext _context;

    public JogadorController(PokeDraftDbContext context) => _context = context;

    [HttpGet]                                           // GET /api/jogador
    public async Task<IActionResult> Listar()
        => Ok(await _context.Jogadores.OrderBy(j => j.Nome).ToListAsync());

    [HttpGet("{id}")]                                   // GET /api/jogador/3
    public async Task<IActionResult> Obter(int id)
    {
        var jogador = await _context.Jogadores.FindAsync(id);
        if (jogador is null) return NotFound();
        return Ok(jogador);
    }

    [HttpPost]                                          // POST /api/jogador
    public async Task<IActionResult> Criar(Jogador jogador)
    {
        if (string.IsNullOrWhiteSpace(jogador.Nome))
            return BadRequest("O nome é obrigatório.");

        _context.Jogadores.Add(jogador);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(Obter), new { id = jogador.Id }, jogador);
    }

    [HttpPut("{id}")]                                   // PUT /api/jogador/3
    public async Task<IActionResult> Atualizar(int id, Jogador dados)
    {
        var jogador = await _context.Jogadores.FindAsync(id);
        if (jogador is null) return NotFound();

        jogador.Nome = dados.Nome;
        await _context.SaveChangesAsync();
        return Ok(jogador);
    }

    [HttpDelete("{id}")]                                // DELETE /api/jogador/3
    public async Task<IActionResult> Remover(int id)
    {
        var jogador = await _context.Jogadores.FindAsync(id);
        if (jogador is null) return NotFound();

        _context.Jogadores.Remove(jogador);
        await _context.SaveChangesAsync();
        return NoContent();
```

```
    }
}
```

Retorno	Código HTTP e quando usar
`Ok(dados)`	200 — deu certo, aqui está o resultado.
`CreatedAtAction(...)`	201 — criado com sucesso.
`NoContent()`	204 — deu certo, não há nada para devolver (típico do DELETE).
`BadRequest("msg")`	400 — o cliente mandou algo inválido.
`NotFound()`	404 — o id não existe.

Passo 3 — O Controller de tela (MVC)

Um controller separado, só para entregar a página. Criar `Controllers/JogadoresController.cs`:

```
public class JogadoresController : Controller
{
    public IActionResult Index() => View();   // devolve Views/Jogadores/Index.cshtml
}
```

> **Por que dois controllers?** Um herda de `ControllerBase` e devolve **JSON** (a API, em `/api/jogador`); o outro herda de `Controller` e devolve **HTML** (a tela, em `/Jogadores`). Separar deixa claro o papel de cada um e evita confusão de rotas.

Passo 4 — A View

Criar `Views/Jogadores/Index.cshtml`:

```
@{
    ViewData["Title"] = "Jogadores";
}

<h1>Jogadores</h1>

<input id="nome" placeholder="Nome do jogador" />
<button onclick="criarJogador()">Adicionar</button>

<ul id="lista"></ul>

@section Scripts {
    <script src="~/js/jogadores.js"></script>
}
```

Passo 5 — O JavaScript

Criar `wwwroot/js/jogadores.js`:

```
async function carregarJogadores() {
    const resposta = await fetch("/api/jogador");
    if (!resposta.ok) {                          // sempre trate o erro
        alert("Erro ao carregar jogadores.");
        return;
    }
    const jogadores = await resposta.json();

    const lista = document.getElementById("lista");
    lista.innerHTML = "";
    jogadores.forEach(j => {
        lista.innerHTML += `<li>${j.nome}
            <button onclick="removerJogador(${j.id})">x</button></li>`;
    });
}

async function criarJogador() {
    const nome = document.getElementById("nome").value;

    const resposta = await fetch("/api/jogador", {
        method: "POST",
        headers: { "Content-Type": "application/json" },   // obrigatório!
        body: JSON.stringify({ nome })
    });

    if (!resposta.ok) { alert("Não foi possível criar."); return; }

    document.getElementById("nome").value = "";
    carregarJogadores();                         // atualiza a lista
}

async function removerJogador(id) {
    await fetch(`/api/jogador/${id}`, { method: "DELETE" });
    carregarJogadores();
}

carregarJogadores();                             // roda ao abrir a página
```

> **Atenção ao nome dos campos no JSON.** No C# a propriedade é `Nome` (maiúscula), mas o ASP.NET converte para `nome` (minúscula) no JSON por padrão. No JavaScript use `j.nome`, não `j.Nome`.

Passo 6 — Testar e commitar

1. `dotnet build` — tem que passar sem erro.
2. `dotnet run --project PokeDraft.Web` e abrir `/Jogadores`.
3. Testar os quatro verbos: listar, criar, atualizar, remover.
4. Abrir o console do navegador (F12) e conferir se não há erro vermelho.
5. `git add`, `git commit`, `git push` e abrir o Pull Request.

8. Convenções do time

Combinar isso agora evita retrabalho e conflito depois.

Assunto	Convenção
Idioma do código	**Português** para tudo que é do domínio (`Jogador` , `Partida` , `PontosGastos`). Termos técnicos que não têm tradução natural ficam em inglês (`Controller` , `Id` , `Overall` , `Sample`).
Classes e propriedades	`PascalCase` : `EscolhaPokemon` , `OrdemNaBatalha` .
Variáveis locais	`camelCase` : `var jogadorAtual = ...`
Campos privados	Prefixo `_` : `private readonly PokeDraftDbContext _context;`
Nome de controller	Sempre termina em `Controller` . API no plural do recurso em `api/[controller]` .
Nome de migration	Descreve a mudança, sem espaços: `AdicionaTabelaNatureza` , `AjustaCustoPokemon` .
Arquivos JS	Um arquivo por tela em `wwwroot/js/` : `jogadores.js` , `draft.js` .
Mensagem de commit	Verbo no infinitivo/passado + o que foi feito, em português. Ex.: `"Adiciona CRUD de Jogador na API"` . Nada de `"update"` ou `"correções"` .
Onde fica regra de negócio	No backend, nunca no JavaScript. Validação no JS é só conveniência — a checagem que vale é a do Controller.

9. Fluxo de trabalho: Git e Trello

9.1 O ciclo de uma tarefa

```
1. Pega um card no Trello e move para "Em desenvolvimento"
2. git pull                                  ← sempre comece atualizado
3. git checkout -b feat/nome-da-tarefa       ← branch própria
4. ... programa ...
5. dotnet build                              ← tem que passar
6. git add .  &&  git commit -m "mensagem clara"
7. git push -u origin feat/nome-da-tarefa
8. Abre Pull Request no GitHub; um colega revisa e aprova
9. Merge na main → move o card para "Concluído"
```

9.2 Regras combinadas

- A `main` tem que **sempre compilar e rodar**. Ninguém commita direto nela.
- Uma tarefa = um branch = um Pull Request. Branches curtos evitam conflitos gigantes.
- Prefixos de branch: `feat/` (funcionalidade nova), `fix/` (correção), `docs/` (documentação).
- Deu `git pull` e veio migration nova? Rode `dotnet ef database update` .
- Vai criar uma migration? **Avise no grupo antes** (ver o alerta na seção 4.6).
- Nunca commite `bin/` , `obj/` ou `pokedraft.db` — o `.gitignore` já cuida disso.

9.3 Trello

O quadro tem um card por aula, com a entrega daquele dia na descrição. Listas: **BackLog** → **Em desenvolvimento** → **Concluído**. Sugestão: quebrar o card da aula em itens de checklist e marcar conforme cada um termina, para todo mundo ver o andamento.

10. Cronograma e divisão de tarefas

Data	Entrega	Envolve mexer em
28/08 ✓	Ambiente, repositório, estrutura da solução, modelagem do banco + 1ª migration	Tudo criado. Concluído.
04/09	Importador da PokéAPI funcionando + CRUD de Pokémon na API	Serviço de importação, `PokemonController`
11/09	CRUD de Jogador, Partida e Configuração + esqueleto do frontend consumindo a API	3 controllers de API, Views, `wwwroot/js/`
18/09	**Marco: protótipo navegável.** Criação de partida e sorteio do pool	`PartidaController`, lógica de sorteio, `PokemonNoPool`, `ItensNoPool`
02/10	Motor do draft no backend: ban, compra, validações, controle de saldo	`DraftController`, `EscolhaPokemon`, `MovimentacaoDraft`
09/10	Venda + trava anti-beco-sem-saída + histórico de movimentações	`DraftController`, `MovimentacaoDraft`
16/10	**Marco: MVP integrado.** Tela do draft completa ponta a ponta	Views, JS, CSS + integração
23/10	Polimento visual, sprites, feedback de erro, responsividade	`site.css`, Views
30/10	**Marco: versão candidata.** Testes, casos de borda, correção de bugs	Tudo
06/11	Congelamento do código, README, roteiro e ensaio da defesa	README, documentação

Sugestão de divisão entre os 4

Frente	Responsabilidade
Dados	Importador da PokéAPI, catálogo de Pokémon e itens, definição dos custos e balanceamento.
Motor do draft	Sorteio dos pools, ban, compra, venda, controle de saldo, validações e histórico.
Batalha	Cálculo do overall, tabela de vantagem de tipagem, simulação dos confrontos e desempate.
Frontend	Telas, CSS, JavaScript de integração com a API, sprites, feedback de erro e responsividade.

As frentes se encontram nas datas de marco (18/09, 16/10, 30/10). Quem terminar antes ajuda a próxima frente — e todo mundo revisa Pull Request de todo mundo.

> **Documento vivo.** Este manual descreve o estado do projeto na 1ª entrega. Sempre que a estrutura mudar de forma relevante (nova entidade, nova pasta, nova convenção), atualize a seção correspondente e gere uma versão nova. O fonte HTML está em `docs/` no repositório.